

ELEC520

Python programming

Summer 2026

객체와 클래스

<https://colab.research.google.com/drive/14FV9ltT8gHrvW5q0QlQzIFCtcADeEDgo?usp=sharing>

객체 지향과 절차 지향

클래스 작성

인스턴스 변수, 메소드, 클래스 변수

__init__, __str__, 스페셜 메소드

캡슐화

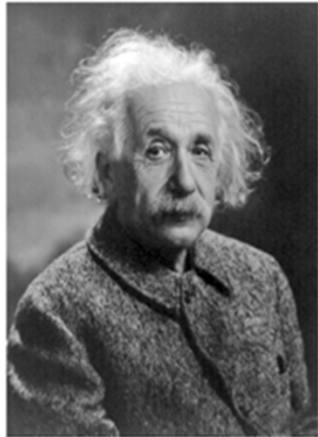
is, is not

mutable / immutable

객체 지향

실제 세계의 객체 object

human



car



tree



- 실제 세계는 객체들로 이루어진다.
- 객체(object)
 - 현실 세계의 사물이나 개념을 시스템에서 이용하기 위해
 - 현실 세계를 자연스럽게 표현하여 손쉽게 이용할 수 있도록 만든 소프트웨어 모델

객체 지향 프로그래밍 OOP

object oriented programming

- 객체 지향 프로그래밍에서는 서로 관련 있는 데이터와 함수를 묶어서 객체(object)로 만들고
- 이들 객체들의 상호작용을 통해 프로그램을 완성한다.

객체object

- 속성과 동작을 가진다.
- 자동차는 메이커나 모델, 색상, 연식, 가격 같은 속성(attribute)을 가지고 있다.
- 또 자동차는 주행할 수 있고, 방향을 전환하거나 정지할 수 있다. 이러한 것을 객체의 동작(action)이라고 한다.

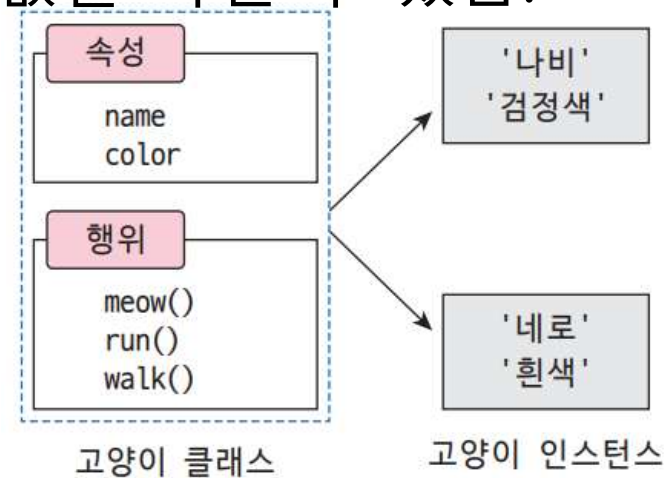
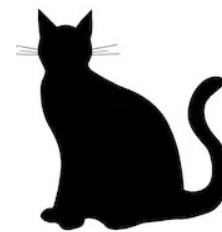
속성
메이커
모델
색상
연식
가격



동작
주행하기
방향바꾸기
정차하기

클래스class

- 객체에 대한 설계도
- 클래스란 특정한 종류의 객체들을 찍어내는 설계도.
- 클래스로부터 만들어지는 개별적인 객체를 그 클래스의 인스턴스 **instance**라고 한다.
- 서로 다른 인스턴스는 서로 다른 속성 값을 가질 수 있음.



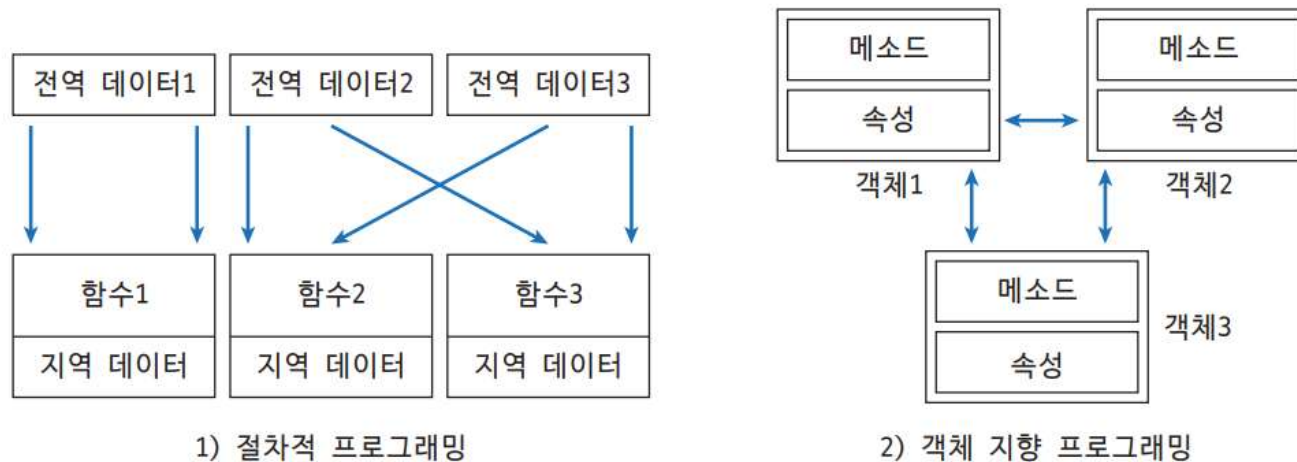
[그림 9-5] 고양이 클래스와 인스턴스 개념도

클래스의 멤버

- 인스턴스 변수 *instance variable*, 멤버 변수 *member variable*, 혹은 필드 *field*
 - 각각의 인스턴스들이 개별적으로 가지는 속성을 저장하는 변수
- 메소드 *method*
 - 클래스안에 존재하는 함수

절차 지향과 객체 지향

- 절차 지향 프로그래밍(procedural programming)은 프로시저 (procedure, 순서, 절차)를 기반으로 하는 프로그래밍 방법이다.
- 객체 지향 프로그래밍(object-oriented programming)은 데이터와 함수를 하나의 덩어리로 묶어서 생각하는 방법이다.

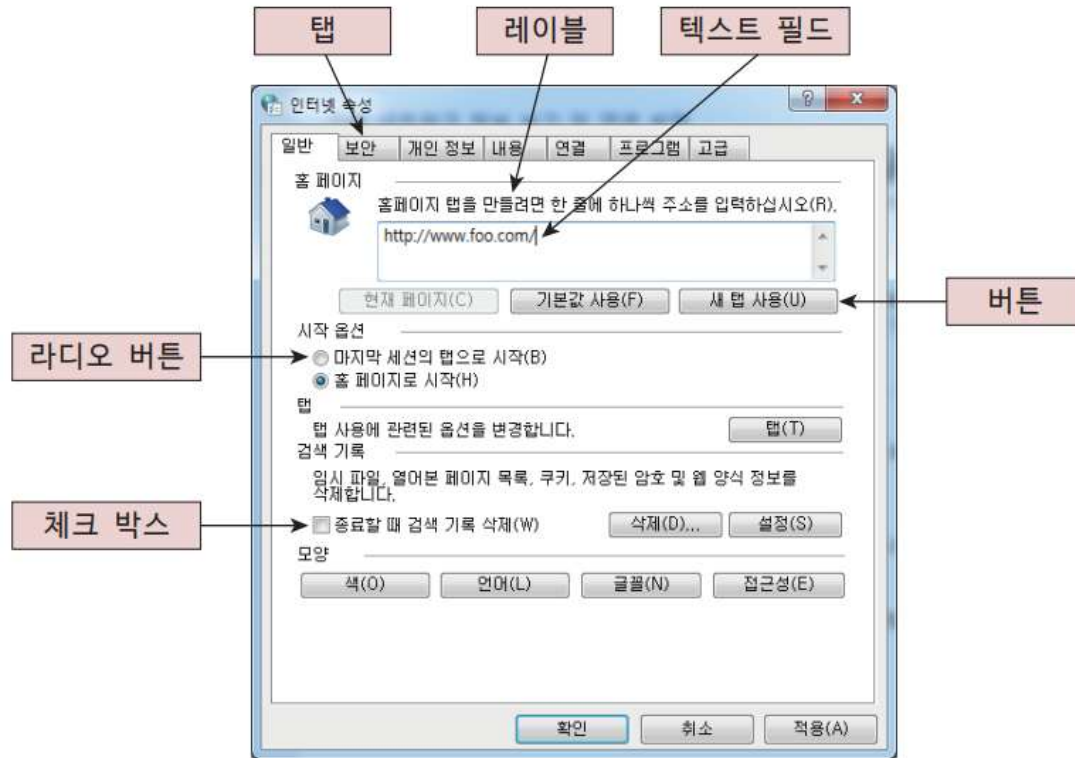


[그림 9-4] 1) 절차적 프로그래밍과 2) 객체 지향 프로그래밍의 비교

절차 지향과 객체 지향(cont.)

- 객체 지향 프로그래밍 **OOP:object oriented programming**
 - 프로그램을 짤 때, 프로그램을 실제 세상에 가깝게 모델링 하는 기법
 - 컴퓨터가 수행하는 작업을 **객체들 사이의 상호작용**으로 표현
 - 클래스 **class**나 객체 **object**들의 집합으로 소프트웨어를 개발하자는 개념
 - Java, Python, C++, C#, Swift 등 현재 사용중인 많은 프로그래밍 언어에서 채택
- 절차적 프로그래밍 언어 **procedural programming language**
 - 함수나 모듈을 만들어두고 이것들을 문제해결 순서에 맞게 호출하여 수행하는 방식
 - C, Fortran, Basic 등의 고전적인 프로그래밍 언어에서 사용함
 - 그래픽 사용자 인터페이스 **graphic user interface** 시스템과 같이 다양한 그래픽 요소들이 있을 경우 효과적으로 문제해결을 하기 힘들다.

아래의 다양한 구성요소(객체)들이 상호작용하도록 하는 것이 객체지향 프로그래밍의 핵심



[그림 9-2] 윈도 운영체제 인터넷 속성창의 나타난 여러 가지 그래픽 객체들



[그림 9-3] 게임 화면과 객체들의 상호 작용

파이썬에서는 모든 것이 객체

- 파이썬에서는 모든 것이 객체로 구현된다.
- 정수도 객체이고 문자열도 객체이며 리스트도 객체

```
>>> type(3)
<class 'int'>
>>> dir(3)
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__dict__', '__dir__', '__divmod__', '__doc__', '__eq__', '__float__', '__floor__', '__floordiv__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__gt__', '__hash__', '__index__', '__int__', '__invert__', '__le__', '__lshift__', '__lt__', '__mod__', '__mul__', '__ne__', '__neg__', '__new__', '__next__', '__or__', '__pos__', '__pow__', '__radd__', '__rand__', '__rdivmod__', '__reduce__', '__reduce_ex__', '__repr__', '__rfloordiv__', '__rmod__', '__rmul__', '__round__', '__rpow__', '__rrshift__', '__rsub__', '__rtruediv__', '__rsplit__', '__rsub__', '__rtruediv__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__truediv__', '__trunc__', '__xor__', 'as_integer_ratio', 'bit_length', 'conjugate', 'denominator', 'imag', 'numerator', 'real', 'to_bytes']
>>> (3).__add__(4)
7
#3+4
```

• int 클래스의 메소드

- __add__()
- __sub__()
- __mult__()
- __truediv__()
- __mod__()
- __pow__()
- __lshift__()
- __rshift__(), ...

이러한 다양한 메소드들을 통해서 산술연산(+, -, *, @, /, //, %, **, <<, >>, &, ^, |) 을 수행한다. 책 2장의 다양한 연산자는 내부적으로 이 메소드의 호출로 동작한다.

리스트도 객체

대화창 실습 : animals 리스트 객체와 다양한 메소드

```
>>> animals = ['lion', 'tiger', 'cat', 'dog']
```

```
>>> animals.sort()
```

```
>>> animals
```

```
['cat', 'dog', 'lion', 'tiger']
```

```
>>> animals.append('rabbit')
```

```
>>> animals
```

```
['cat', 'dog', 'lion', 'tiger', 'rabbit']
```

```
>>> animals.reverse()
```

```
>>> animals
```

```
['rabbit', 'tiger', 'lion', 'dog', 'cat']
```

- 파이썬의 **리스트**^{list}는 'lion', 'tiger', 'cat', 'dog' 등의 항목을 원소(**속성**)로 가질 수 있음.
- 또한 sort(), append(), remove(), reverse(), pop() 라는 함수(**메소드**)를 가지고 있음(**. 표기로 메소드 호출**)



NOTE : Everything in Python is an object.

우선 다음과 같은 코드를 다시 한 번 살펴보자.

```
'tiger'.upper()
animals.append('rabbit')
```

문자열은 upper()라는 메소드로 대문자 변환을 수행

리스트는 append()라는 메소드로 새 요소를 추가함

이 코드에서 사용한 'tiger'라는 문자열과 animals라는 리스트는 upper(), append()와 같은 메소드를 통해서 지정된 일을 수행할 수 있는데 이러한 프로그래밍의 구성 요소를 **객체object**라고 한다. 파이썬은 객체 지향 프로그래밍 언어로 여기에서 사용되는 데이터는 모두 객체이다.

그렇다면 다음과 같은 수식과 연산에 사용되는 n이나 100, 200은 객체일까?

```
n = 100 + 200
```

수식 연산 역시 아래의 코드와 같이 메소드 호출로 동작함

그렇다. 위의 코드는 다음 코드와 동일하다.

```
n = (100).__add__(200)
```

즉, 위의 코드에서 100이라는 정수 객체는 __add__() 메소드를 이용해서 200이라는 객체를 더하는 일을 한다. 그리고 이 객체는 n이라는 변수가 참조하여 접근하게 된다. 이런 점에서 더하기(+) 연산자 역시 본질적으로 animals.append('rabbit')이라는 메소드 호출 코드와 수행 절차가 동일하다.



LAB 9-1 : 객체와 메소드 호출

1. 다음 메소드 호출의 결과는 무엇인가?

```
>>> (200).__sub__(100)
a) _____
>>> (200).__mul__(100)
b) _____
>>> (200).__truediv__(100)
c) _____
```

2. 다음 메소드 호출의 결과는 무엇인가?

```
>>> [10, 20, 30, 40].pop()
a) _____
```

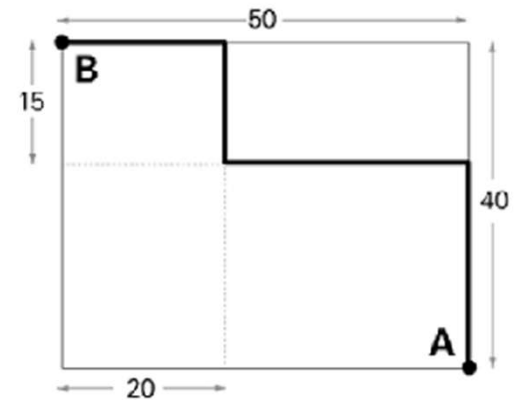
3. 다음 중 리스트 객체가 호출할 수 없는 메소드는 무엇인가?

1) pop() 2) keys() 3) remove() 5) get()

클래스 작성

클래스 작성과 객체 생성 연습

- 직진, 좌/우회전, 정지를 할 수 있는 자동차
 - 자동차 객체 생성하여 A지점에서 B지점까지 네비게이션 경로를 따라 주행 테스트
- 강아지
 - 속성: 이름, 나이, 포만감
 - 먹고, 말하고, 놀 수 있는 강아지
 - 먹기
 - 음식 x 를 주면 " x 맛있어요~"하고 포만감 2 증가
 - 포만감이 10 이상이면 "배불러서 안 먹을래요."
 - 말하기
 - 멍멍
 - 놀기
 - 장난감 x 를 받아 " x 가지고 놀이하니 재미있어요."



클래스 작성

Syntax: 클래스 정의

형식

```
class 클래스이름 :  
    def __init__(self, ...) :  
        ...  
    def 메소드1(self, ...) :  
        ...  
    def 메소드2(self, ...) :  
        ...
```

예

```
class Counter:  
    def __init__(self):  
        self.count = 0  
    def increment(self):  
        self.count += 1
```

생성자를 정의한다.

메소드를 정의한다.

```
class Counter:  
    def __init__(self):  
        self.count = 0  
    def increment(self):  
        self.count += 1  
  
a = Counter()  
a.increment()  
print("카운터의 값=", a.count)
```

- 인스턴스의 메소드 호출

인스턴스이름.메소드([인자])

- nabi라는 인스턴스는 Cat이라는 클래스가 가진 meow()라는 메소드를 사용할 수 있음

코드 9-3 : Cat 클래스 정의와 여러 개의 객체 생성

many_cats.py

```
class Cat:
```

```
    def meow(self):
```

```
        print('야옹 야옹~~~')
```

```
nabi = Cat()
```

```
nabi.meow()
```

```
nero = Cat()
```

```
nero.meow()
```

```
mimi = Cat()
```

```
mimi.meow()
```

nabi 객체가 meow()실행

nero 객체가 meow()실행

mini 객체가 meow()실행

실행결과

야옹 야옹~~~

야옹 야옹~~~

야옹 야옹~~~

`__init__()`, `__str__()`, `__repr__()`

- `__init__()`
 - **생성자**
 - 객체를 만들 때 인스턴스 내부의 변수가 기본값을 가지도록 하는 역할을 하는 메소드
 - 객체가 생성될 때 자동으로 실행
- `__str__()`
 - 객체의 정보를 **문자열로 반환하는 메소드**
 - 보통은 객체의 정보를 표현하도록 재정의
 - 구현되지 않을 경우 `__repr__`의 내용이 반환
- `__repr__()`
 - 객체의 정보를 반환하는 메소드(디버깅용)

```
>>> st = 'hello'
>>> st
'hello'
>>> print(st)
hello
```

코드 9-5 : __str__ 메소드와 print() 함수에서 적용하기

__str__()

str_cats.py

```
class Cat:
    def __init__(self, name, color):
        self.name = name
        self.color = color

    # Cat 객체의 문자열 표현방식
    def __str__(self):
        return 'Cat(name='+self.name+', color='+self.color+)'

nabi = Cat('나비', '검정색')      # nabi 인스턴스 생성
nero = Cat('네로', '흰색')       # nero 인스턴스 생성
print(nabi)
print(nero)
```

실행결과

```
Cat(name=나비, color=검정색)
Cat(name=네로, color=흰색)
```

Special method

벡터의 합 구하기

코드 9-11 : 사용자 정의 Vector2D 클래스와 + 연산

vector2d_error.py

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

v1 = Vector2D(30, 40)
v2 = Vector2D(10, 20)
v3 = v1 + v2
print('v1 + v2 = ',v3)  # Vector2D의 + 연산이 정의되지 않았다:오류 출력
```

실행결과

TypeError: unsupported operand type(s) for +: 'Vector2D' and 'Vector2D'

- 에러가 나타나는 이유는 클래스 내부에 어떤 방식으로 덧셈을 할지 그 방법을 서술해야 하기 때문
- add()라는 메소드를 정의하고 두 벡터의 x 성분과 y 성분을 더하고 이 성분값을 초기값으로 가지는 Vector2D를 반환하도록 해보기

코드 9-12 : add() 메소드를 이용한 벡터의 덧셈

vector_add.py

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __str__(self):
        return "({}, {})".format(self.x, self.y)
    def add(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)

v1 = Vector2D(30, 40)
v2 = Vector2D(10, 20)
v3 = v1.add(v2)          # Vector2D의 add() 메소드 사용
print('v1.add(v2) =', v3)
```

실행결과

v1.add(v2) = (40, 60)

add() 구현

- v1.add(v2)는 두 벡터의 합을 출력
- add() 메소드 대신에 +, -와 같은 연산자를 사용하면 보다 편리

코드 9-13 : `__add__`, `__sub__`, `__str__` 특수 메소드와 연산자의 관

vector2d_add_sub.py

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)
    def __sub__(self, other):
        return Vector2D(self.x - other.x, self.y - other.y)
    def __str__(self):
        return "({}, {})".format(self.x, self.y)
```

```
v1 = Vector2D(30, 40)
v2 = Vector2D(10, 20)
v3 = v1 + v2
print('v1 + v2 =', v3)
v4 = v1 - v2
print('v1 - v2 =', v4)
```

실행결과

```
v1 + v2 = (40, 60)
v1 - v2 = (20, 20)
```

`__add__()` 구현

- `__add__()`에는 두 벡터의 합을 구하는 기능을 구현하고, `__sub__()`에는 두 벡터의 차를 구하는 기능을 구현
- `v1 + v2`는 `v1.__add__(v2)`와 같은 메소드 호출과 동일한 기능

스페셜 메소드

- 형태: `__메소드이름__()`
- 보통은 직접적으로 호출되기보다 간접적으로 호출되는 메소드
 - `__init__()`
 - `__str__()`
 - `__len__()`
 - ...
- `len()`이라던지 `float()`, `int()`, `str()`, `abs()`, `hash()`, `iter()`과 같이 많은 내장 함수들은
 - `__len__()`, `__float__()`, `__int__()`, `__str__()`, `__abs__()`, `__hash__()`, `__iter__()`과 같은 특수 메소드로 구현이 가능

연산자와 스페셜 메소드

연산자	특수 메소드	하는 일
$x + y$	<code>__add__(self, other)</code>	x와 y의 합을 구한다.
$x - y$	<code>__sub__(self, other)</code>	x와 y의 차를 구한다.
$x * y$	<code>__mul__(self, other)</code>	x와 y의 곱을 구한다.
$x ** y$	<code>__pow__(self, other)</code>	x의 y 거듭제곱을 구한다.
x / y	<code>__truediv__(self, other)</code>	x를 y로 나눈 값을 구한다.
$x // y$	<code>__floordiv__(self, other)</code>	x를 y로 나눈 몫을 구한다.
$x \% y$	<code>__mod__(self, other)</code>	x를 y로 나눈 나머지를 구한다.
$+x$	<code>__pos__(self)</code>	x를 구한다.
$-x$	<code>__neg__(self)</code>	x의 음수를 구한다.

연산자와 스페셜 메소드(cont.)

연산자	특수 메소드	하는 일
x < y	__lt__(self, other)	x가 y보다 작은가?
x <= y	__le__(self, other)	x가 y보다 작거나 같은가?
x >= y	__ge__(self, other)	x가 y보다 크거나 같은가?
x > y	__gt__(self, other)	x가 y보다 큰가?
x == y	__eq__(self, other)	x와 y가 같은가?
x != y	__ne__(self, other)	x와 y가 서로 다른가?



LAB 9-8 : 특수 메소드의 응용

1. 앞서 배운 `__mul__()`과 `__truediv__()` 메소드를 이용하여 두 벡터의 곱셈과 나눗셈 기능을 구현하여라. `v1`이 (30, 40)이고 `v2`가 (10, 20)이라고 가정하고 다음과 같은 결과가 나타나도록 출력문을 작성하여라.

```
v1 * v2 = (300, 800)
v1 / v2 = (3.0, 2.0)
```

2. 앞서 배운 `__neg__()` 메소드 이용하여 벡터의 음의 벡터를 구하시오. `v1`이 (10, 20)일 경우 출력 값은 다음과 같다.

```
-v1 = (-10, -20)
```



LAB 9-9 : 벡터의 크기 비교하기

1. 앞서 배운 비교 연산자에 해당하는 특수 메소드를 이용하여 두 벡터의 크기를 비교하는 프로그램을 작성하시오. `v1`이 (30, 40)이고 `v2`가 (10, 20)이라고 가정하면 다음과 같은 결과가 나타나도록 출력문을 작성하여라.

```
v1 > v2 = True
v1 >= v2 = True
v1 < v2 = False
v1 <= v2 = False
```

캡슐화

코드 9-6 : nabi.age에 직접 값을 할당하기

cat_age_change.py

```
class Cat:
    def __init__(self, name, age):
        self.name = name
        self.age = age
        # Cat 객체의 문자열 표현방식

    def __str__(self):
        return 'Cat(name='+self.name+', age='+str(self.age)+')'

nabi = Cat('나비', 3)          # nabi 인스턴스 생성
print(nabi)
nabi.age = 4
nabi.age = -5
print(nabi)
```

age가 음수가 되는
비정상적 상황

인스턴스 변수 직접 접근

실행결과

```
Cat(name=나비, age=3)
Cat(name=나비, age=-5)
```

문법 오류는 아니지만 값이
보호받지 못한 상태의 논리
적 문제

캡슐화encapsulation

- 인스턴스 변수를 외부에서 함부로 조작하는 것을 제한
- 데이터를 보호
- 잘못된 값으로 멤버 변수 값을 설정하려는 경우 사전 차단

캡슐화와 getter, setter

코드 9-7 : set_age() 메소드를 통해서 age 값을 할당하기

cat_age_with_setter_getter.py

```
class Cat:
    def __init__(self, name, age):
        self.__name = name
        self.__age = age
    # Cat 객체의 문자열 표현방식
    def __str__(self):
        return 'Cat(name='+self.__name+', age='+str(self.__age)+')'
    # self.__age를 외부에서 자유롭게 접근하는 것을 제한하고 음수가 되지 않도록 함
    def set_age(self, age):
        if age > 0:
            self.__age = age
    def get_age(self):
        return self.__age

nabi = Cat('나비', 3) # nabi 인스턴스 생성
print(nabi)
nabi.set_age(4)      # set_age() 메소드를 통해서 age에 접근
nabi.set_age(-5)     # set_age() 메소드를 통해서 age가 음수가 되지 않도록 함
print(nabi)
```

- if 조건식을 넣어서 age 값이 음수일때는 할당이 되지 않도록 해보기
- setXXX와 같이 시작하는 메소드를 세터setter라고 함
- 반대로 getXXX와 같이 시작하는 게터getter를 통해서 멤버 값을 읽어오는 것도 가능

실행결과

Cat(name=나비, age=3)
Cat(name=나비, age=4)

나이가 음수가 되는 비논리적 상황을 해결하는 방법

클래스 변수

코드 9-14 : Circle 인스턴스와 속성 PI

circle_instance_var.py

```
class Circle:
    def __init__(self, name, radius, PI):
        self.name = name
        self.radius = radius
        self.PI = PI

    # 현재 인스턴스의 PI에 반지름**2 을 곱하여 면적을 구함
    def area(self):
        return self.PI * self.radius ** 2

c1 = Circle("C1", 4, 3.14)
print("c1의 면적:", c1.area())
c2 = Circle("C2", 6, 3.141)
print("c2의 면적:", c2.area())
c3 = Circle("C3", 5, 3.1415)
print("c3의 면적:", c3.area())
```

실행결과

```
c1의 면적: 50.24
c2의 면적: 113.076
c3의 면적: 78.53750000000001
```

클래스 변수

- 인스턴스 변수들 중에서 클래스가 공유해야 할 공통적인 속성이 있을 것이다.
- Circle 클래스의 self.PI 값이 일관성 있게 유지가 되려면, c1, c2, c3에서 PI 값을 모두 공유할 필요성이 있다. 이때, 사용하는 것이 **클래스 변수 class variable**이다.

클래스 변수(cont.)

- Circle의 클래스 변수 pi는 이 클래스의 인스턴스들이 공유하는 변수이며, 클래스 속성 `__name`, `__radius`는 인스턴스마다 가지는 변수인 인스턴스 변수이다.

- pi 값을 더욱 정밀한 값으로 변경해야 한다면 간단하게 클래스 변수 `PI = 3.141592`와 같이 변경해주기만 하면 된다.

class Circle:

```
    PI = 3.1415 # 클래스 변수
    def __init__(self, name, radius):
        self.__name = name # 인스턴스 변수
        self.__radius = radius # 인스턴스 변수
```

Circle 클래스의 변수 :
인스턴스들은 이 변수를 공유함

Circle 클래스의 속성 :
인스턴스마다 가지는 변수

- PI 값을 더욱 정밀한 값으로 변경해야 한다면 간단하게 클래스 변수 $PI = 3.141592$ 와 같이 변경해주기만 하면 된다.

코드 9-15 : Circle 인스턴스와 클래스 변수 PI

circle_class_var.py

```
class Circle:
    PI = 3.1415          # 클래스 변수
    def __init__(self, name, radius):
        self.__name = name      # 인스턴스 변수
        self.__radius = radius  # 인스턴스 변수
    # Circle 클래스의 변수 PI를 이용하여 면적을 구함
    def area(self):
        return Circle.PI * self.__radius ** 2

c1 = Circle("C1", 4)
print("c1의 면적:", c1.area())
c2 = Circle("C2", 6)
print("c2의 면적:", c2.area())
c3 = Circle("C3", 5)
print("c3의 면적:", c3.area())
```

iS vs. ==

객체와 참조, 할당 연산의 의미

대화창 실습 : n = 100의 할당 연산과 객체의 id 값

```
>>> n = 100
```

```
>>> id(100)
```

```
4509016272
```

```
>>> id(n)
```

```
4509016272
```

- 변수명이란 객체에 대한 참조명
 - 100이라는 객체를 n이라는 이름의 변수를 통해서 접근
- 각 객체의 아이디(id)를 출력해 보기

- 객체를 참조하는 변수 n과 m은 새로운 객체를 참조할 수도 있다
 - 최초상태의 n은 100을 참조하고 있으며, m = n을 통해 m도 역시 동일한 객체를 참조하고 있음
 - n = 200을 통해 m과 n이 다른 객체를 참조함

대화창 실습 : 할당 연산자를 통한 참조와 n = 200 연산자를 이용한 재참조

```
>>> n = 100
```

```
>>> m = n
```

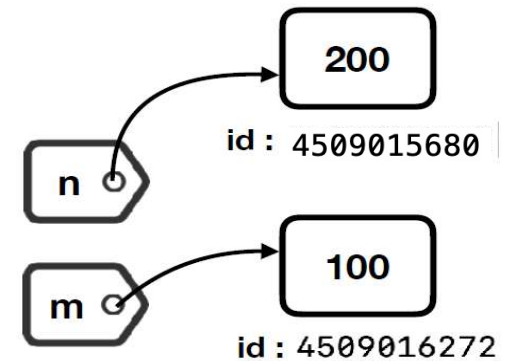
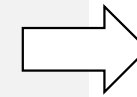
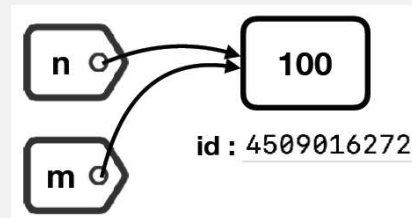
```
>>> n = 200
```

```
>>> id(n)
```

```
4509015680
```

```
>>> id(m)
```

```
4509016272
```



- $n = n + 1$ 의 연산자의 연산과 객체 할당 방법
- $n = n + 1$ 을 통해 새롭게 할당된 n 과 이전의 n 은 서로 다른 객체를 참조한다는것을 확인함

대화창 실습 : $n = n + 1$ 연산과 객체의 id 값의 변화

```
>>> n = 100
```

```
>>> id(n)
```

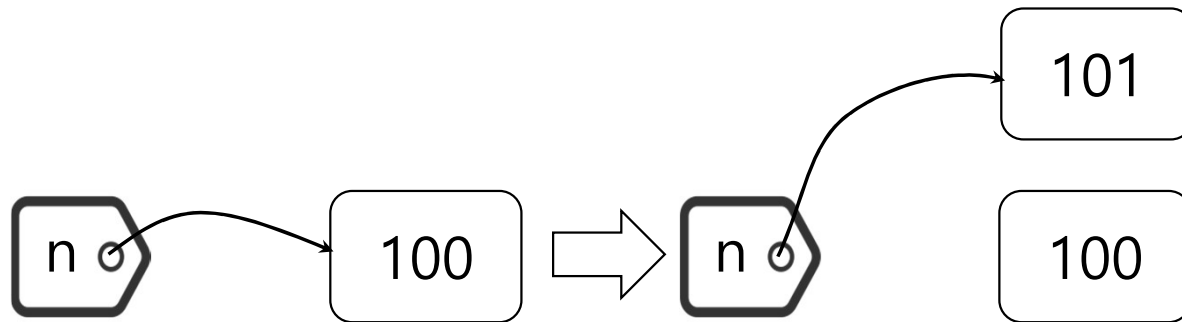
```
4509016272
```

```
>>> n = n + 1
```

```
>>> id(n)    # 할당 연산에 의해 n이 새로운 객체를 참조하고 있음
```

```
4509016346
```


- n이 참조하고 있는 정수형 객체가 **변경 불가능**immutable 속성으로 정의되어 있음



- 더 이상 이 객체를 참조하는 변수가 존재하지 않으므로 메모리 낭비를 초래
- 메모리상에 존재하는 객체 중에서 참조가 사라져서 더 이상 참조할 방법이 없는 객체를 **가비지**garbage 라고 부르며
- 주기적으로 정리하는 메모리 관리 절차를 **가비지 수집**garbage collection이라고 한다.

객체의 아이덴티티 연산 identity operator : is, is not

- is 연산자는 두 인스턴스가 같으면(id 가 같으면) True를 반환하며, 그렇지 않으면 False를 반환. is not은 반대 역할을 함

코드 9-8 : 객체의 아이덴티티 연산자 : is, is not 연산자

is_test.py

```
list_a = [10, 20, 30]          # 리스트 객체를 참조하는 list_a
list_b = [10, 20, 30]          # 리스트 객체를 참조하는 list_b

if list_a is list_b:           # 두 리스트 객체가 같은지 검사함
    print('list_a is list_b')
else:
    print('list_a is not list_b')
```

실행결과

list_a is not list_b

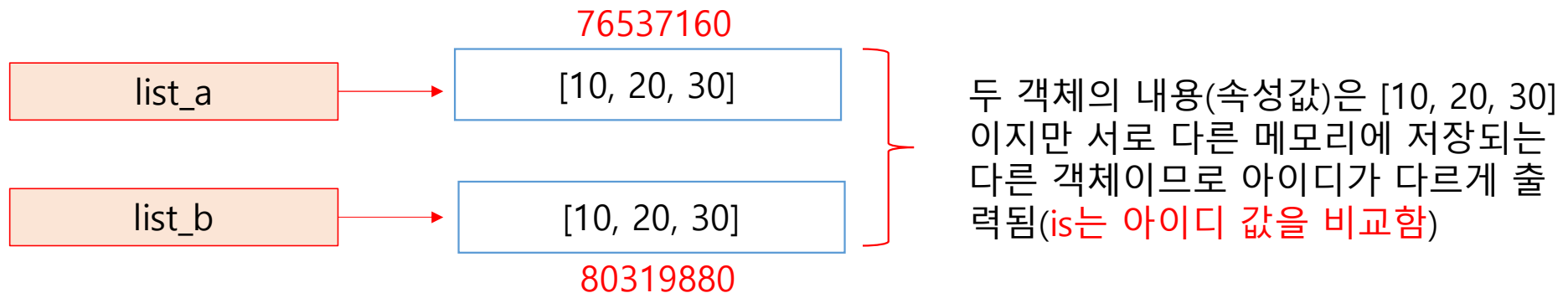
mutable 객체에서 is

- id(list_a), id(list_b)를 출력하는 방법으로 결과 확인

```
print('id(list_a) =', id(list_a), ', id(list_b) = ', id(list_b))
```

```
id(list_a) = 76537160 , id(list_b) = 80319880
```

- 출력 결과 list_a와 list_b는 각기 다른 아이디 값을 가짐



list_a is list_b 는 False

is vs. ==

- == 연산자는 두 인스턴스의 속성 값 즉 인스턴스 변수 값이 서로 일치하는지 확인하는데 사용

코드 9-9 : 두 개의 리스트와 == 연산

equal_test.py

```
list_a = [10, 20, 30]          # 리스트 객체를 참조하는 list_a
list_b = [10, 20, 30]          # 리스트 객체를 참조하는 list_b

if list_a == list_b:           # 리스트 객체의 속성 값이 같은지 비교함
    print('list_a == list_b')
else:
    print('list_a != not list_b')
```

실행결과

list_a == list_b

76537160

[10, 20, 30]

==

[10, 20, 30]

80319880

list_a = list_b 는 True

immutable 객체에서 is

코드 9-10 : 두 개의 문자열 참조변수와 is, is not 연산

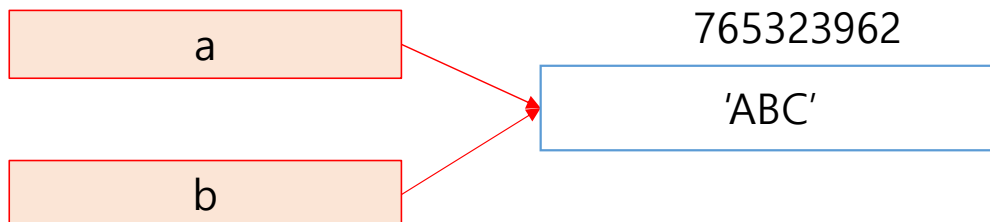
equal_string_test.py

```
a = 'ABC'           # 문자열 객체를 참조하는 변수 a
b = 'ABC'           # 문자열 객체를 참조하는 변수 b
if a is b:          # 문자열 객체 a, b가 같은가 비교
    print('a is b')  # 문자열 객체 a, b는 같은 객체를 참조함
else:
    print('a is not b')
```

문자열은 불변객체(immutable) 이므로
이 방법을 사용해도 아무런 문제가
발생하지 않는다

실행결과

a is b



파이썬의 str 객체는 문자 객체를 저장
하는 표에 넣어 놓은 후 a, b에 같은 문
자를 할당할 경우 **메모리를 절약**하기
위해서 같은 저장위치를 참조한다

a와 b가 같은 객체를 참조함



LAB 9-7 : 정수 객체의 is 연산

1. 다음 코드의 수행 결과는 무엇인가? 결과를 적고 그 이유를 설명하여라.

```
n = 100
m = 100
if n is m:
    print('n is m')
else:
    print('n is not m')
```



NOTE : 불변 속성을 가진 객체의 참조

정수, 실수, 문자열, 부울, 튜플 객체는 불변형 객체이다. 따라서 다음과 같이 할 경우에 m, n은 동일한 객체 100을 참조한다.

```
>>> n = 100
>>> m = 100    # m = n과 동일한 결과
```

이 때문에 이 식의 수행 결과는 [그림 9-13]의 결과와 동일하다. 이와 같이 하는 이유는 메모리를 효율적으로 사용하기 위해서이다.

Remark

- 객체(실체), 클래스(설계도)
 - 데이터와 그 데이터로 할 수 있는 동작을 한 곳에 모아 둔 것.
- 함수
 - 특정 기능을 수행하는 코드 모음
- 메소드
 - 특정 객체에서 수행할 수 있는 기능